

CUSTOMER JOURNEY MAP

Credit Card Fraud Detection System Using Machine Learning

Prepared by: **Bhargavasai** • Internship Project Showcase

SCENARIO

A user or evaluator wants to check whether a specific credit card transaction is legitimate or fraudulent using a real-time ML prediction demo. The web application features a responsive HTML/CSS frontend, a robust Python Flask backend proxying model execution, an advanced machine learning pipeline (XGBoost/Random Forest) integrated via Joblib, and is fully deployed on Render cloud architecture.

DIMENSION	ENTICE Discover the project	ENTER Start prediction	ENGAGE Fill transaction data	ENGAGE Submit to ML API	EXIT Read prediction result	EXTEND Review codebase
Steps Typical user experiences	Visits the home page. Reads that the tool provides instant, automated fraud predictions to eliminate manual human error.	Clicks the navigation link to head to the prediction page. Navigates to the input form interface.	Enters the specific transaction details required by the model: Time, V1 to V28 PCA features, and Amount .	Clicks the Predict button. The frontend sends the structured transaction fields to the backend server.	Views the final real-time verdict text: either "Normal Transaction" or "Fraud Transaction Detected."	Opens the GitHub Repository to inspect the source code, trained model files, testing reports, and README.
Interactions Touchpoints used	Home page UI, navigation bar, project objective section, and feature cards.	Prediction page routing link, primary Call-to-Action (CTA) button.	Structured input form, numerical text fields for Time/Amount, and 28 feature boxes.	Predict submit button, Flask backend routing, Python request handler, and saved Joblib model object.	Result display card, alert status banner, real-time response interface.	GitHub repository, README .md, project documentation, testing reports, and public Render URL.
People & Things Who/what is involved	Bhargavasai (Presenter), evaluator/demo user, and the web app home page.	User, web browser, and the web routing layer.	Transaction dataset features, browser input elements, and layout validation.	Flask API server, trained ML model file (.pk1), Pandas DataFrame, and Render cloud platform.	User, browser output window, and Flask rendering engine.	GitHub version control platform, source code, data analysis scripts, and public live link.

DIMENSION	ENTICE Discover the project	ENTER Start prediction	ENGAGE Fill transaction data	ENGAGE Submit to ML API	EXIT Read prediction result	EXTEND Review codebase
Goals Help the user...	Understand the core purpose and how automation replaces slow, manual fraud checks.	Easily transition from introductory pages to the functional testing interface without friction.	Provide all the necessary mathematical transaction features accurately without validation errors.	Route input parameters to the backend smoothly, trigger the ML model, and process a rapid calculation.	Clearly understand if the tested transaction exhibits high-risk fraudulent patterns or is safe.	Review the end-to-end engineering pipeline including EDA, model selection, evaluation metrics, and cloud deployment.

POSITIVE MOMENTS

- **Instant Automation:** Replaces error-prone manual transaction analysis with real-time predictions.
- **Streamlined Flow:** Seamless data passage between HTML/CSS frontend and Python Flask backend.
- **Robust Engineering:** Full pipeline including handling missing values, duplicate removal, and `StandardScaler` normalization.
- **Public Accessibility:** Live cloud deployment on Render enables testing from any device without environment configuration.

NEGATIVE MOMENTS

- **Input Complexity:** Manually typing 28 abstract PCA components (V1-V28) can feel overwhelming for non-technical users.
- **Instance Latency:** Free cloud hosting tier (Render) introduces potential spin-up delays during initial requests.
- **Imbalance Sensitivity:** High data skew inherent to fraud requires extremely precise threshold adjustments to prevent false negatives.

AREAS OF OPPORTUNITY

- **Explainable AI (XAI):** Embed SHAP or LIME visualizations inside the Flask UI to detail *why* a transaction was flagged.
- **Batch CSV Processing:** Add a file upload widget to let evaluators upload a batch of records rather than inputting single items.
- **Live Dashboard:** Integrate an interactive model metric panel (Plotly/Dash) showing confusion matrices and accuracy logs in real time.